crypto-steganography: one file, two directions

main() and setup_argparse()
hide or extract, three words on the command line

HIDE: hide_message(image, message, password)

salt = os.urandom(16), fresh each hide

encrypt_message: Fernet token

bits: salt 128, length 32, payload

_write_bits sets one bit per pixel,

channel cycles red, green, blue

hidden_<name>.png, always PNG
JPEG recompresses and kills the bits

EXTRACT: extract_message(image, password)

_read_bits reads the segments back

in the same fixed order

salt first, then the 32 bit length,

then that many payload bits

decrypt_message: Fernet open

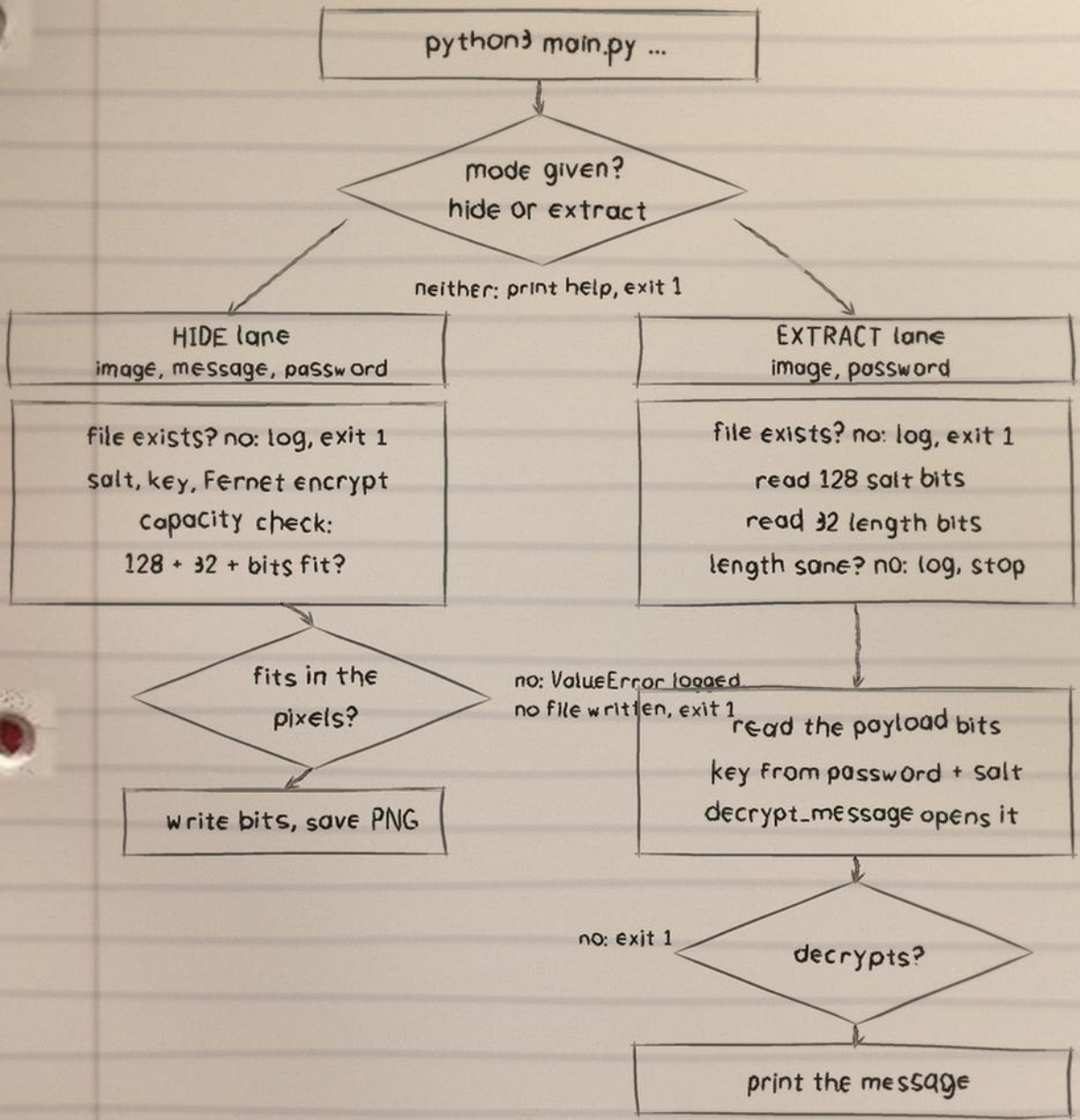
print: Extracted message: ...
wrong password logs a failure, prints nothing

generate_key(password, salt), shared by both sides
PBKDF2-HMAC-SHA256, 390000 rounds, 32 byte key, base64

One bit per pixel is the whole capacity: width x height bits, minus the 160 bit header.
main.py is the only code file. Pillow opens the image; cryptography does the cipher work.

cover label, signed GR

A run, followed: hide on the left, extract on the right



Both lanes end the same way on failure: a log line, no output, and exit code 1. Success on hide is a new PNG in the current folder, nothing else.

Product map: a secret that rides inside a picture

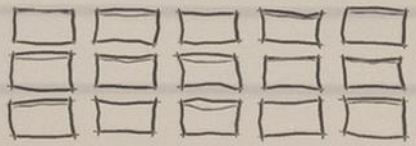
WHO IT SERVES

Me, sending a short text that should not sit in plain sight, and anyone who needs the same quiet trick.

INPUTS

Hide: an image file, the message, a password.
Extract: the PNG that came out, the same password.

THE PICTURE



pixels carry one bit each

OUTPUTS

One new PNG named hidden_<name>.png, or on extract the message itself, printed to the terminal.

BOUNDARIES

Text only, one image at a time. No batches, no folders, no network. A JPEG copy of the output loses the secret.

under the pixels: a Fernet token, so the hidden text is ~~cipher text first~~

Hide, as data: text to cipher to bits in a row

password + fresh salt

↓

generate_key: PBKDF2-HMAC-SHA256, 390000 rounds

↓

encrypt_message: the text becomes a Fernet token

then the file is written as three segments, one bit per pixel, in this fixed order:

1. SALT	2. LENGTH	3. PAYLOAD
128 bits, from os.urandom(16)	32 bits, the payload bit count	the Fernet token, byte by byte, 8 bits each

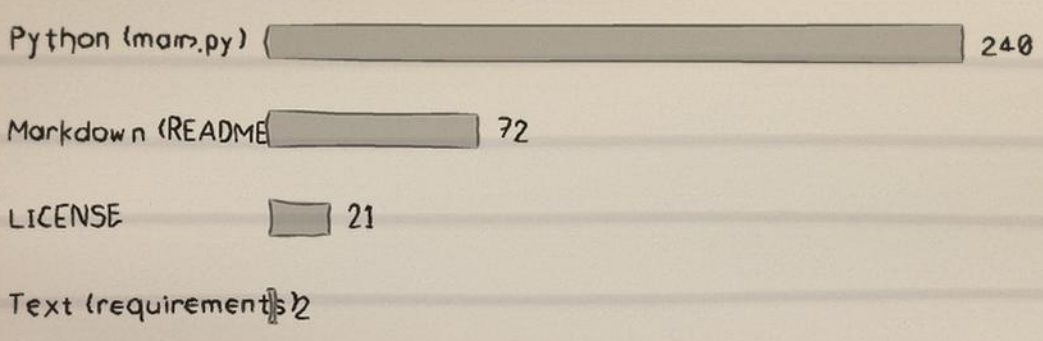
Extract walks the same three strips in reverse: salt, length, payload, then the key, then the text.

Each segment restarts the red, green, blue channel cycle at red. Both sides depend on that.

traced from main

Crypto-steganography in numbers, counted 2026-10-30

Lines by kind, from git ls-files and a line count over each tracked text file. The two PNG brand images are excluded.



The whole repo is 6 files and 8 commits. First commit 2025-03-28, last one 2026-10-30, both from git log.
Tests: none in the repo. The README says the round trip was verified by hand; I ran no suite here.
Full table, with the counting method behind every number, in VISUAL.md in this folder.

counted and drawn, GK

Crypto Steganography: schematic of the apparatus

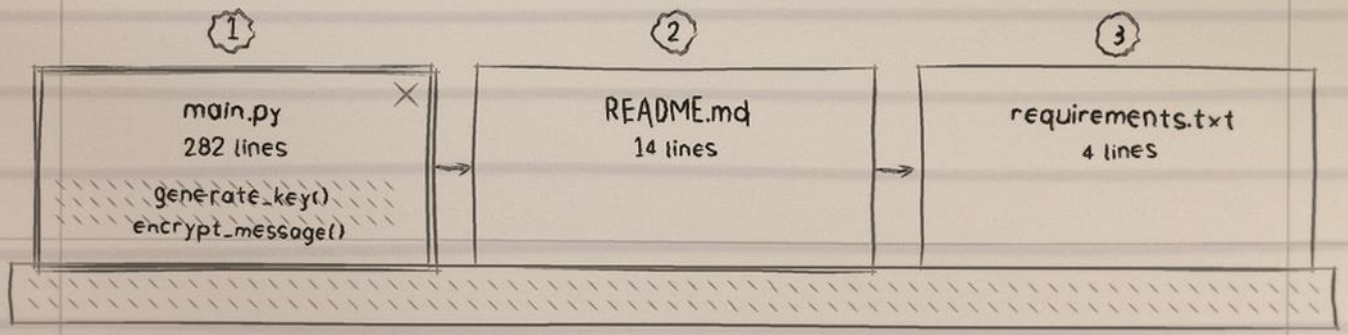
the system as a machine: parts, flow, and measurements

construction lines left in, as drawn

parts list

1. main.py
2. README.md
3. requirements.txt

in flow order, first to last



base plate: the repo itself, sectioned

span: 3 files, 300 lines counted

measured in the code: loop cap 128 (main.py)

turns of the crank so far: 4 commits, 2025-03-28 to 2025-03-28 measured and sketched, GK